## Complete finite certificate for the projective string-marking theory

The complete required program is printed below, including the literal source correction witnesses. It is charged once within the topic. The separate finite sample validation program is not a premise of the uniform written proofs.

```
# Complete finite proof for the common projective string-marking theory.
# Reworks the prior six-local/ten-universal census and the
source-correction witnesses.
# GAP's finitely presented group algorithms are retained computational
machinery.
orb:=[]; iso:=fail; vertices:=[];
Fail := function(msg) Error(msg); end;
Need := function(b,msg) if not b then Fail(msg); fi; end;
IP := function(G, rr)
  local ss, hs, a,b;
  ss:=Combinations([1..Length(rr)]);
  hs:=List(ss,s->Subgroup(G,rr{s}));
  for a in [1..Length(ss)] do for b in [1..Length(ss)] do
    if
    Intersection(hs[a],hs[b])<>hs[Position(ss,Intersection(ss[a],ss[b]))]
    then return false; fi;
  od; od; return true;
end;
# The small isomorphisms used in the written applications.
for pp in [[PSL(2,2),SymmetricGroup(3)],[PSL(2,3),AlternatingGroup(4)],
           [PGL(2,3),SymmetricGroup(4)],[PSL(2,5),AlternatingGroup(5)],[P
           GL(2,5),SymmetricGroup(5)]] do
  Need(IsomorphismGroups(pp[1],pp[2])<>fail,"small projective-group
  identification");
od;
# Every ordered string marking; IP is checked only AFTER admission.
for pair in [[SymmetricGroup(4),[[3,3,4],[3,4,3],[4,3,3]]],
             [AlternatingGroup(5),[[3,5,5],[5,3,5],[5,5,3]]]] do
  G:=pair[1]; inv:=Filtered(Elements(G),x->Order(x)=2); markings:=[];
  for a in inv do for c in inv do if a<>c and a*c=c*a then
    for b in inv do rr:=[a,b,c];
      if Length(Set(rr))=3 and Subgroup(G,rr)=G then
        Add(markings,rr);
        Need(Order(a*b)>2 and Order(b*c)>2,"all-sggi adjacent orders");
        Need(IP(G,rr),"all-sggi local intersection property");
      fi;
    od;
  fi; od; od;
  if Size(G)=24 then expected:=72;else expected:=360;fi;
  Need(Length(markings)=expected,"complete all-sggi local census");
  aut:=AutomorphismGroup(G); remaining:=ShallowCopy(markings); types:=[];
  while Length(remaining)>0 do
    rr:=remaining[1]; orb:=Orbit(aut,rr,OnTuples);
    Add(types,[Order(rr[1]*rr[2]),Order(rr[2]*rr[3]),Order(Product(rr))])
    ;
    remaining:=Filtered(remaining,x->not x in orb);
  od;
  Sort(types); Need(types=pair[2],"exceptional rank-three marking
  census");
  F3:=FreeGroup("x","y","z"); z3:=GeneratorsOfGroup(F3);
  for typ in types do
    R3:=Concatenation(List(z3,a->a^2),[(z3[1]*z3[3])^2,
```

```
      (z3[1]*z3[2])^typ[1],(z3[2]*z3[3])^typ[2],Product(z3)^typ[3]]);
    Need(Size(F3/R3)=Size(G),"complete rank-three presentation");
  od;
  Print("RANK3 ",Size(G)," MARKINGS ",Length(markings)," TYPES
  ",types,"\n");
od;
# [facet m,n,k; vertex m,n,k; universal order; string IP; face counts]
rows := [
 [[5,5,3],[5,5,3],1,false,[]],
 [[5,5,3],[5,3,5],1,false,[]],
 [[5,3,5],[3,5,5],3420,true,[57,171,171,57]],
 [[5,3,5],[3,4,3],60,false,[]],
 [[5,3,5],[3,3,4],1,false,[]],
 [[4,3,3],[3,4,3],96,true,[4,6,6,4]],
 [[4,3,3],[3,3,4],24,false,[]],
 [[3,5,5],[5,3,5],660,true,[11,55,55,11]],
 [[3,4,3],[4,3,3],1,false,[]],
 [[3,3,4],[3,3,4],120,true,[5,10,10,5]] ];
localtypes:=[[3,3,4],[3,4,3],[4,3,3],[3,5,5],[5,3,5],[5,5,3]];
allpairs:=Filtered(Cartesian(localtypes,localtypes),p->p[1][2]=p[2][1]);
revtype:=t->[t[2],t[1],t[3]];
revpair:=p->[revtype(p[2]),revtype(p[1])];
covered:=Set(Concatenation(List(rows,r->[r[1],r[2]]),List(rows,r->revpair
([r[1],r[2]]))));
Need(Length(allpairs)=14 and Set(allpairs)=covered,"exhaustive compatible
local pairs");
Need(Length(Filtered(allpairs,p->p=revpair(p)))=6,"six self-dual local
pairs");
F:=FreeGroup("r0","r1","r2","r3"); x:=GeneratorsOfGroup(F);
for n in [1..Length(rows)] do
  row:=rows[n]; f:=row[1]; v:=row[2]; Need(f[2]=v[1],"shared edge
  label");
  rel:=List(x,a->a^2);
  Append(rel,[(x[1]*x[2])^f[1],(x[2]*x[3])^f[2],(x[3]*x[4])^v[2],
              (x[1]*x[3])^2,(x[1]*x[4])^2,(x[2]*x[4])^2,
              (x[1]*x[2]*x[3])^f[3],(x[2]*x[3]*x[4])^v[3]]);
  U:=F/rel; Need(Size(U)=row[3],"fp universal order"); if row[3]=1 then
  G:=Group(()); rr:=[(),(),(),()]; else iso:=IsomorphismPermGroup(U);
  Need(iso<>fail,"fp permutation isomorphism"); G:=Image(iso);
  rr:=List(GeneratorsOfGroup(U),a->Image(iso,a)); fi;
  Need(Size(G)=row[3],Concatenation("universal order row ",String(n)));
  good:=ForAll(rr,a->Order(a)=2) and Length(Set(rr))=4 and IP(G,rr);
  Need(good=row[4],"rank-four intersection property");
  if good then
    faces:=List([1..4],i->Index(G,Subgroup(G,rr{Difference([1..4],[i])}))
    );
    Need(faces=row[5],"face counts");
  else faces:=[]; fi;
  if n in [3,8,10] then
    if n=3 then target:=PSL(2,19); elif n=8 then target:=PSL(2,11); else
    target:=SymmetricGroup(5); fi;
    Need(IsomorphismGroups(G,target)<>fail,"universal group
    identification");
    dual:=GroupHomomorphismByImages(G,G,rr,Reversed(rr));
    Need(dual<>fail and IsBijective(dual),"self-duality");
  fi;
  if n in [4,7] then
```

```
    if n=4 then target:=AlternatingGroup(5); else
    target:=SymmetricGroup(4); fi;
    Need(IsomorphismGroups(G,target)<>fail,"collapsed group
    identification");
    Need(Subgroup(G,rr{[1,2,3]})=G or
    Subgroup(G,rr{[2,3,4]})=G,"collapsed full facet or vertex figure");
  fi;
  if n=6 then
    V:=FittingSubgroup(G); Need(Size(V)=16 and
    IsElementaryAbelian(V),"row-six elementary normal subgroup");
    Need(IsomorphismGroups(FactorGroup(G,V),SymmetricGroup(3))<>fail,"row
    -six quotient");
    Need(Length(ComplementClassesRepresentatives(G,V))>0,"row-six
    splitting");
  fi;
  if n=8 then
    vertices:=RightCosets(G,Subgroup(G,rr{[2,3,4]}));
    edges:=RightCosets(G,Subgroup(G,rr{[1,3,4]}));
    ends:=List(edges,e->Filtered([1..11],i->not
    IsEmpty(Intersection(Set(e),Set(vertices[i])))));
    Need(ForAll(ends,e->Length(e)=2) and Length(Set(ends))=55,"11-cell
    complete edge graph");
  fi;
  if n=6 then
    Need(Order(rr[1]*rr[2])=4 and Order(rr[3]*rr[4])=4,"order96 outer
    labels are4 not Petrie3");
    for indices in [[1,2,3],[2,3,4]] do
      H:=Subgroup(G,rr{indices});
      Need(Size(H)=24 and H<>G and
      IsomorphismGroups(H,SymmetricGroup(4))<>fail,"proper S4 ends in
      parity sharpness example");
    od;
  fi;
  Print("ROW ",n," ORDER ",Size(G)," IP ",good," FACES ",faces,"\n");
od;

StringMarking:=function(rr)
 local i,j;
 if Length(Set(rr))<>Length(rr) or not ForAll(rr,x->Order(x)=2) then
 return false;fi;
 for i in [1..Length(rr)] do for j in [i+1..Length(rr)] do
 if j>i+1 and rr[i]*rr[j]<>rr[j]*rr[i] then return false;fi;
 od;od;return true;
end;
 # Exact counterexample to literal source Lemma3.3 maximality.
 G:=AlternatingGroup(5);z:=(1,2)(3,4);H:=Stabilizer(G,5);cs:=Centralizer(
 G,z);
 Need(Size(cs)=4 and Size(H)=12 and IsSubgroup(H,cs) and
 H<>G,"centralizer maximality counterexample");
 Print("SOURCE_L33_COUNTEREXAMPLE PSL5 CENTRALIZER4 CONTAINED_A4_12\n");
 # The smallest omitted S4 example is literal, avoiding a subgroup
 search.
 rr:=[(1,2)(3,24)(4,23)(5,22)(6,21)(7,20)(8,19)(9,18)(10,17)(11,16)(12,15
 )(13,14),
 (1,5)(2,7)(3,21)(4,11)(6,17)(8,23)(9,13)(10,24)(12,14)(15,18)(16,19)(20,
 22),
 (1,4)(2,23)(3,9)(5,13)(6,16)(7,10)(8,15)(11,21)(12,19)(14,22)(17,20)(18,
 24),
```

```
(1,10)(2,17)(3,22)(4,18)(5,24)(6,7)(8,13)(9,23)(11,15)(12,16)(14,19)(20,
21)];
G:=PSL(2,23);Need(ForAll(rr,x->x in G) and Group(rr)=G and
StringMarking(rr),"literal PSL23 marking");
H:=Subgroup(G,rr{[1..3]});Need(Size(H)=24 and
IsomorphismGroups(H,SymmetricGroup(4))<>fail,"PSL23 S4 end");
Need(Subgroup(G,rr{[2..4]})=G and Index(G,H)=253,"omitted253facet
count");
Need(List([1..3],i->Order(rr[i]*rr[i+1]))=[3,4,12],"literal
omitted-family type");
Print("OMITTED_S4 PSL23 ORDER6072 FACETS253 VERTICES1 TYPE3,4,12
PASS\n");
# A literal self-dual Class1 marking and its actual degree14 conjugator.
rr:=[(3,9)(4,10)(5,11)(6,12)(7,13)(8,14),(1,4)(2,14)(3,9)(5,13)(6,11)(7,
12),
(1,2)(3,7)(4,6)(8,14)(9,13)(10,12),(1,2)(4,14)(5,13)(6,12)(7,11)(8,10)];
G:=PSL(2,13);dual:=(1,3)(2,9)(4,7)(5,8)(6,12)(10,11)(13,14);
Need(ForAll(rr,x->x in G) and Group(rr)=G and StringMarking(rr),"literal
selfdual marking");
Need(Subgroup(G,rr{[1..3]})=G and Subgroup(G,rr{[2..4]})=G,"selfdual
Class1 ends");
Need(List([1..3],i->Order(rr[i]*rr[i+1]))=[3,7,3] and
List(rr,x->x^dual)=Reversed(rr),"literal selfdual reversal");
Need(List(rr,x->x^(dual*(1,2)))<>Reversed(rr),"altered conjugator
rejected");
Print("SELFDUAL_CLASS1 PSL13 ORDER1092 TYPE3,7,3 CONJUGATOR_PASS
ALTERED_REJECTED\n");

Print("TOPIC_E_REQUIRED_FINITE_PASS\n");
QUIT;
```